DRP: Distilled Reasoning Pruning with Skill-aware Step Decomposition for Efficient Large Reasoning Models

Yuxuan Jiang[1] **Dawei Li**[2] **Francis Ferraro**[1]
[1]University of Maryland, Baltimore County
[2]Arizona State University
yuxuanj1@umbc.edu

Abstract

While Large Reasoning Models (LRMs) have demonstrated remarkable success in complex reasoning tasks through Long Chain-of-Thought (CoT) reasoning, their inference often involves excessively verbose reasoning traces, resulting in substantial inefficiency. To address this issue, we propose **Distilled Reasoning Pruning (DRP)**, a hybrid framework that combines inference-time pruning with tuning-based distillation—two widely used strategies for efficient reasoning. DRP employs a teacher model to perform *skill-aware* step decomposition and content pruning, and then distills the pruned reasoning paths into a student model, enabling it to reason both efficiently and accurately. Across a series of challenging mathematical reasoning datasets, we find models trained with DRP achieve substantial improvements in token efficiency without sacrificing accuracy. Specifically, DRP reduces the average token usage on GSM8K from 917 to 328 while improving accuracy from 91.7% to 94.1%, and achieves a 43% token reduction on AIME with no performance drop. Further analysis reveals that aligning the reasoning structure of training CoTs with the student's reasoning capacity is critical for effective knowledge transfer and performance gains. We release our code at: https://github.com/YuxuanJiang1/DRP

1 Introduction

Although Large Reasoning Models (LRMs) (Xu et al., 2025a), like OpenAI's o1 (OpenAI, 2024b) and DeepSeek-R1 (Guo et al., 2025), have advanced the state of the art in complex reasoning tasks (Li et al., 2025c), a critical limitation of these models is their tendency toward *overthinking*—the generation of excessively verbose reasoning trajectories containing redundant or unnecessary steps (Chen et al., 2024; Cui et al., 2025; Fu et al., 2024). This can lead to substantial inference overhead and misguide the model toward incorrect conclusions (Sui et al., 2025).

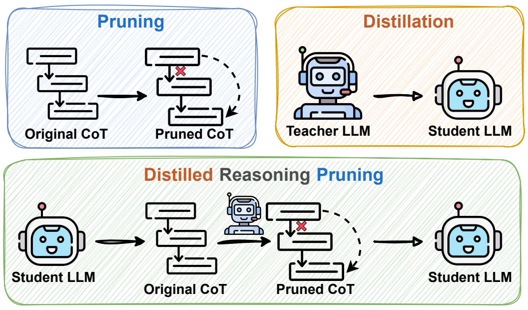

Figure 1: An overview of our proposed Distilled Reasoning Pruning framework, which unifies pruning and distillation by having the teacher model prune the student model's reasoning chains and distill the resulting concise CoT back into the student. This approach improves reasoning efficiency without sacrificing accuracy.

Existing solutions predominantly follow two paradigms: *inference-time pruning*, which attempts to terminate generation early to avoid redundant reasoning steps (Cui et al., 2025; Muennighoff et al., 2025; Fu et al., 2024; Zeng et al., 2025), and *distillation-based compression*, where smaller models are trained on teacher-generated reasoning paths to imitate the concise reasoning behavior of larger models (Tan et al., 2024; Zhu et al., 2024; Xu et al., 2025b). However, both approaches can be at the cost of accuracy: pruning methods risk prematurely halting the reasoning process, thereby preventing the model from reaching the correct answer, while distillation methods tend to underperform—particularly for small-scale student models—due to the learnability gap, which hinders effective knowledge transfer when the performance or size gap between student and teacher is too great (Xu et al., 2024; Li et al., 2025b).

To address these limitations, we propose **Distilled Reasoning Pruning (DRP)** (Fig. 1), a hybrid framework that combines the strengths of both pruning and distillation (student/teacher)

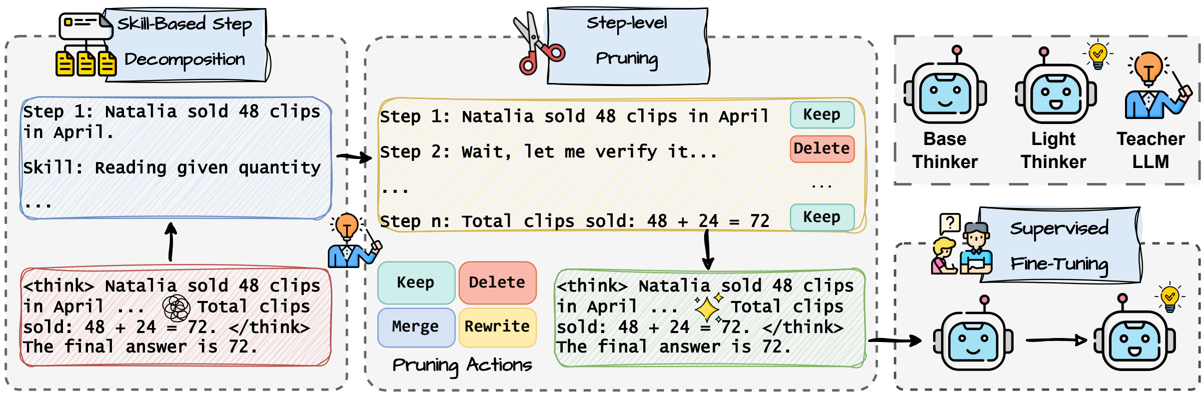

Figure 2: Overview of our proposed DRP framework, which distills concise, skill-aware reasoning paths into a target model to improve reasoning efficiency while preserving accuracy.

paradigms. In particular, rather than simply distilling down a response from the teacher model, we use the teacher model to prune an initial, lengthy, CoT reasoning trajectory from the student model. To facilitate this, and to encourage shorter yet informative resulting pruned trajectories, we introduce a **skill-based step decomposition** method. The teacher model uses this to prune the trajectory, which produces more stable and semantically coherent reasoning units. Unlike prior methods that rely solely on either teacher-generated (Xu et al., 2025b; Zhu et al., 2024) or self-sampled (Chen et al., 2024; Ma et al., 2025) concise trajectories for distillation, DRP takes advantage of the teacher's pruning within the student model's original reasoning structure. This design reduces the learnability gap and enables student models to achieve efficient reasoning without compromising performance.

Consider the example in Fig. 2. For this word problem, the student model generates its initial CoT trajectory (shown in the <think> block). This trajectory is provided to a teacher model, which segments that CoT into steps, with a high-level description of the *skill* that that step demonstrates or exercises. The teacher model then prunes and curates these skill-segmented steps, such as by merging similar ones (if the student was redundant or verbose) or deleting steps (e.g., for backtracking). This pruned CoT trajectory is then provided to the student model for supervised fine-tuning.

We conduct extensive experiment in various student models, teacher models and mathematical datasets. DRP consistently improves the efficiency and accuracy of student models—even on significantly harder out-of-distribution tasks. Our ablation study demonstrates that DRP substantially outperforms direct distillation approaches. In addition, we show that our framework consistently benefits from various teacher models. We believe all these findings and insights from DRP can benefit future works in efficient reasoning in LRMs. Our key contributions are as follows:

- We propose **Distilled Reasoning Pruning**, a novel framework that unifies step-level pruning and distillation to improve both reasoning efficiency and accuracy in small-scale LLMs.

- We introduce a **skill-based step decomposition** method that segments reasoning traces into semantically coherent and functionally aligned units, providing a stable foundation for pruning and supervision.

- We point out that effective training CoTs should be both informative and structurally consistent with the student's reasoning process, thereby facilitating knowledge transfer and bridging the capability gap between teacher and student models.

2 Related Work

2.1 The Overthinking Problem

Large reasoning models, such as OpenAI o1 (OpenAI, 2024b) and DeepSeek-R1 (Guo et al., 2025), are a specialized subclass of large language models trained to iteratively generate intermediate reasoning steps and refine the thinking process until reaching a final answer (Chen et al., 2025; Sui et al., 2025; Yu et al., 2025). Unlike traditional LLMs, which rely on test-time strategies such as Chain-of-Thought prompting to elicit detailed reasoning (Tong et al., 2024), LRMs are trained to internalize CoT generation. This design significantly enhances their performance on reasoning-intensive tasks, particularly in challenging mathematics and programming. However, LRMs, especially those with smaller parameter sizes, often exhibit a tendency to produce overly verbose reasoning chains, including unnecessary tracebacks and redundant alternative paths (Chen et al., 2024; Sui et al., 2025; Cui et al., 2025; Fu et al., 2025, 2024). Such *overthinking* not only increases token consumption beyond practical inference budgets but also introduces reasoning drift that can degrade output correctness (Hou et al., 2025; Sui et al., 2025). To address these, our method explicitly prunes harmful overthinking behaviors while simultaneously guiding the model toward concise and effective reasoning expressions.

2.2 Token-Efficient Reasoning Methods

Current token-efficient reasoning methods can be categorized into three approaches (Sui et al., 2025):

(1) **Prompt-based methods** impose token budget constraints within the prompt to encourage concise reasoning without altering model parameters. For example, TALE (Han et al., 2024) estimates per-instance token budgets to reduce length while maintaining accuracy on simple tasks. However, these methods rely on handcrafted prompt designs and struggle with complex inputs. (2) **Supervised fine-tuning (SFT)** approaches train models on compressed reasoning traces to promote internalized efficiency (Xia et al., 2025; Munkhbat et al., 2025). CoT-Valve (Ma et al., 2025), for instance, uses iterative compression stages to fine-tune reasoning conciseness. Yet, these approaches often require task-specific data and retraining. (3) **Reinforcement learning (RL)** methods guide models toward concise reasoning by introducing explicit reward signals that penalize overly long outputs (Team et al., 2025; Chen et al., 2024). Some approaches incorporate early-exit mechanisms at inference time (Muennighoff et al., 2025; Fu et al., 2024; Zeng et al., 2025). A representative method is ThinkPrune (Hou et al., 2025), which defines a target reasoning length and progressively fine-tune models under increasingly tighter constraints.

Despite progress, existing approaches often trade off accuracy, especially in out-of-domain (OOD) tasks, for brevity. In contrast, we propose a lightweight framework that leverages external teacher models to perform *skill-aware pruning* over reasoning steps. This structure-aware intervention enables significant token reduction while improving performance, especially under distribution shift.

3 Methodology

We propose **D**istilled **R**easoning **P**runing (DRP), a method that distills concise reasoning paths into a student model by first decomposing them into skill-based steps and then pruning redundant content with the help of a teacher model (e.g., GPT-4o). As illustrated in Figure 2, DRP proceeds in three stages: (1) decomposing the original reasoning into fine-grained, skill-based steps; (2) pruning and rewriting these steps via a teacher LLM; and (3) fine-tuning the student model on the revised, compact trajectories. These stages result in token-efficient supervision that improves both reasoning efficiency and accuracy during fine-tuning.

3.1 Skill-Based Step Decomposition

To solve a math problem, a reasoning model generates a token sequence as the full response, which consists of two components: a structured thinking process T, enclosed within the <think> </think> tags, and a final answer summarization A, captures the conclusive statement derived from the reasoning trace. Together, we terms these a "response," $R = (T, A)$. For instance, in example shown in Fig. 2, the "thinking" portion of the initial response would include "Natalia sold 48 clips in ... Total clips sold: 48 + 24 = 72" while the answer summarization is "The answer is 72."

We extract the thinking segment T from the student model's response and prompt a teacher model to decompose it into a sequence of non-overlapping segments, each aligned with a functional reasoning skill such as arithmetic computation, comparison, or logical inference. This skill-based decomposition serves as the foundation for step-level pruning and distillation. Compared to naïve sentence-based splitting, it enhances the *stability* of step boundaries, maintains *semantic coherence* across reasoning steps, and provides a more *fine-grained supervision signal* (in §6.1). We validate the quality of this segmentation through a pairwise evaluation using Gemini 2.0 Flash as an external judge; results and examples are presented in Appendix D for review.

Formally, given a response $R = (T, A)$, we define a decomposition function D that maps the thinking trace into skill-annotated steps:

$$D(T) \mapsto \{(s_1, k_1), (s_2, k_2), \ldots, (s_m, k_m)\},$$

where each s_i denotes a contiguous span of tokens representing a single reasoning step, and k_i is the associated skill label, inferred by the teacher model based on its internal knowledge and understanding of the reasoning process. For the example in Fig. 2, one segment should be "Natalia sold 48 clips in April," which corresponding to the skill "Reading given quantity." However, these skills can cover broader topics such as "Algebraic representation" and "Interpret a fraction of a subset." The complete decomposition prompts are provided in Appendix C.

3.2 Step-Level Pruning

For each reasoning step–skill pair (s_i, k_i), the teacher model is prompted to revise the step without altering the essential structure or logical intent of the original reasoning. The teacher selects one of the following actions for each step:

- **Keep:** Retain the step unchanged.
- **Delete:** Remove the step if it is redundant or uninformative.
- **Rewrite:** Replace the step with a more concise version conveying the same logic.
- **Merge:** Combine the step with adjacent ones if they form a coherent atomic unit.

This yields a revised step $\hat{s}_i = \text{Revise}(s_i)$, and a pruned reasoning trace: $\hat{T} = \{\hat{s}_1, \hat{s}_2, \ldots, \hat{s}_{m'}\}$, where $m' \leq m$, which reduces redundancy and increases the overall information density.

Finally, the teacher model rewrites $\hat{T}$ into a fluent, coherent reasoning trace that preserves the tone and speaking style of the student model. To ensure consistency between the revised reasoning and the final answer, we prompt the teacher to optionally revise the original answer segment A, yielding an updated final answer summarization $\hat{A}$. The final output becomes a concatenation $\hat{R} = (\hat{T}, \hat{A})$, which we use as the target for supervised fine-tuning. Complete prompt are in Appendix C.

Self-Revision Prompt

Given a list of reasoning steps labeled with their respective skills, your task is to evaluate and revise each step according to one of the four ACTIONs.
Ensure the resulting reasoning path is concise, fluent, and logically sound. At the end, synthesize the revised steps into a coherent explanation that matches the speaker's tone.

3.3 Supervised Fine-Tuning

We construct the training dataset using pairs $(x, \hat{R})$, where x is the input question and $\hat{R}$ is the complete revised response.

We fine-tune the model using teacher-forced decoding to encourage the generation of concise reasoning traces. The training objective minimizes the negative log-likelihood of the revised response:

$$\mathcal{L}_{\text{SFT}} = -\sum_{i=1}^{n} \log P_\theta(y_i \mid x, y_{<i}),$$

where $\{y_1, \ldots, y_n\}$ are the tokens in $\hat{R}$, and θ denotes the model parameters. This supervision enables the model to internalize skill-aligned, token-efficient reasoning strategies while maintaining consistency with the final answer.

4 Experimental Setup

4.1 Datasets

Our training corpora consist of the training split of GSM8K (Cobbe et al., 2021) and the full PRM12K (Lightman et al., 2023) dataset. From these, we generate initial reasoning paths, which are then processed through skill-based step decomposition and teacher-guided step-level pruning to create supervision signals for fine-tuning. To evaluate complex mathematical reasoning and generalization ability, we select a broad set of out-of-domain benchmarks including MATH500 (Hendrycks et al., 2021), AIME24 (AI-MO Team, 2024a), and AMC23 (AI-MO Team, 2024b).

4.2 Models

We use DeepSeek-R1-Distill-Qwen-7B and DeepSeek-R1-Distill-Qwen-1.5B (Guo et al., 2025) as student models for supervised fine-tuning. Both are distilled variants of DeepSeek-R1 optimized for efficient inference. Our primary

Method	GSM8K		MATH500 (OOD)		AIME24 (OOD)		AMC (OOD)	
	Pass@1	#Tokens	Pass@1	#Tokens	Pass@1	#Tokens	Pass@1	#Tokens
R1-Distill-Qwen-1.5B								
Base	70.7%	1443	80.4%	3276	6/30	10484	23/40	6516
+TALE	70.1%	1170	76.2%	3107	6/30	8915	22/40	6235
+Cot Valve	70.4%	805	76.5%	2705	7/30	**5601**	22/40	4574
+Thinkprune	80.0%	**712**	79.2%	**2006**	9/30	5745	25/40	**3291**
+DRP	**83.4%**	721 (-50%)	**82.0%**	2122 (-35%)	**10/30**	6135 (-42%)	**27/40**	3657 (-44%)
R1-Distill-Qwen-7B								
Base	91.7%	917	92.4%	2486	15/30	8674	31/40	4845
+TALE	91.0%	522	91.6%	2530	10/30	8602	31/40	3998
+Cot Valve	90.8%	364	89.4%	1975	13/30	6315	30/40	**3157**
+DRP	**94.1%**	**328** (-64%)	**93.0%**	**1781** (-28%)	**15/30**	**4966** (-43%)	**33/40**	3258 (-33%)

Table 1: Pass@1 accuracy and average token usage on R1-Distill-Qwen models across various math benchmarks, comparing our DRP method with Cot Valve (Ma et al., 2025), TALE (Han et al., 2024), and ThinkPrune (Hou et al., 2025).

teacher model is GPT-4o (OpenAI, 2024a), which performs step-level decomposition and pruning to generate supervision signals. For ablation studies, we additionally explore alternative teacher models, including Gemini 2.0 Flash (DeepMind, 2025) and ChatGPT (OpenAI, 2022).

4.3 Compared Methods

We select three representative methods that span the major paradigms for token-efficient reasoning: (1) **TALE** (Han et al., 2024): a prompt-based method incorporates a soft token budget constraint into the prompt to encourage concise generation. (2) **CoT-Valve** (Ma et al., 2025): a SFT-based method which enerates multiple chains-of-thought of varying lengths for the same problem, and performs supervised fine-tuning in multiple rounds—each time using shorter CoTs as training targets. (3) **ThinkPrune** (Hou et al., 2025): a tuning-based method uses reinforcement learning to iteratively reduce chain-of-thought length by optimizing under a target token constraint.

4.4 Implementation Details

Supervised fine-tuning is performed using the LLaMA-FACTORY[1] framework with LoRA adaptation. All models are trained for 3 epochs with cosine learning rate scheduling. Full hyperparameters are listed in Appendix B.

[1]https://github.com/hiyouga/LLaMA-Factory

4.5 Evaluation Protocol

We use the LM-EVALUATE-HARNESS[2] framework for unified evaluation across tasks. Each model is evaluated in a zero-shot setting. For inference, we use the vLLM backend with the maximum generation length set to 131,072 tokens—the upper limit supported by the Qwen models.

4.6 Evaluation Metrics

We report **Pass@1** as the accuracy metric. For efficiency, we measure the number of reasoning tokens generated per completion using the HuggingFace-compatible Qwen tokenizer.[3]

We also observe that models occasionally fall into degenerate loops, repeatedly generating parts of their responses until reaching the maximum generation limit (e.g., 130k tokens), far exceeding the typical average length (e.g., 5k). In most of these cases, the model fails to answer correctly. Such outliers significantly inflate the average token count.

To mitigate their impact, we set a cutoff threshold of 12k tokens, which empirically covers 99% of correct responses across all benchmarks for the models we evaluate. Detailed token length distributions by task are provided in Appendix A.

5 Main Results

5.1 Accuracy and Token Efficiency

Table 1 presents the main results on our DRP and other compared methods. Our key findings are:

[2]https://github.com/EleutherAI/lm-evaluation-harness

[3]https://huggingface.co/Qwen/Qwen-tokenizer

DRP consistently reduces token usage across all benchmarks and model sizes. Our proposed DRP method achieves substantial reductions in average token usage on both in-domain and out-of-domain tasks. On GSM8K, DRP reduces token count by up to 64% with the 7B model. For out-of-domain datasets, DRP yields 28%–44% reductions across all benchmarks, demonstrating strong generalization beyond the training distribution.

DRP improve the accuracy by mitigating the over-thinking problem in LRMs. Despite significantly reducing token usage, DRP preserves or improves Pass@1 accuracy on nearly all benchmarks. Notably, DRP improves accuracy even on harder datasets such as AMC and MATH500. The only exception is AIME24 under the 7B setting, where accuracy remains unchanged, suggesting the inherent difficulty of this benchmark.

Accuracy gains are more pronounced for smaller models. DRP delivers particularly strong improvements with the 1.5B model. On GSM8K, accuracy increases by 12.7%, while on AIME24 and AMC, DRP answers 4 more problems correctly compared to the base model. This indicates DRP's effectiveness in compensating for limited model capacity through more efficient supervision.

5.2 Comparison to Prior Work

We compare our approach with three representative baselines covering prompt-based, SFT-based, and RL-based token-efficient reasoning strategies (details can be found in Section 4.3).

Prompt-based methods offer limited control on Complex Reasoning Tasks. TALE is simple to implement and demonstrates moderate effectiveness on short-answer tasks such as GSM8K. It achieves token reductions with minimal accuracy degradation (less than 1%) on both model sizes. However, its effectiveness diminishes significantly on more challenging benchmarks. For example, on AIME24 with the 7B model, TALE causes a notable performance drop. Overall, TALE's prompt constraints provide limited control over fine-grained reasoning behaviors, which we hypothesize leads to its suboptimal performance on tasks requiring deeper or longer reasoning chains. In contrast, our DRP method remains effective even on such difficult tasks.

Existing SFT Methods struggle to balance accuracy and efficiency. CoT-Valve achieves consistent token reductions across benchmarks by training on compressed CoT. However, this often comes with accuracy trade-offs. For instance, on MATH500 and AMC, we observe accuracy degradation despite reduced token usage. By comparison, our DRP method achieves both stronger compression and accuracy improvements across most benchmarks. Notably, on AMC (1.5B), DRP improves accuracy from 22/40 to 27/40 while also reducing tokens from 4574 to 3657.

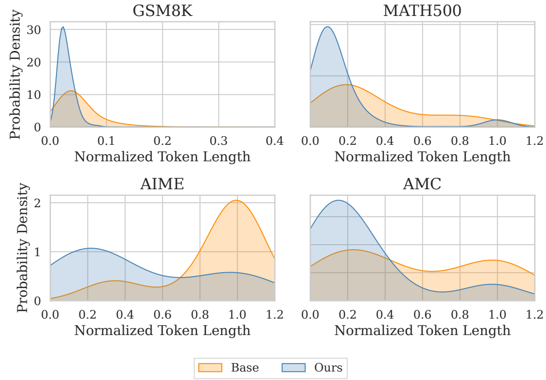

Figure 3: Normalized token length distributions across GSM8K, MATH500, AIME24, and AMC before and after SFT using the DeepSeek-R1-Distill-Qwen-7B model. The horizontal axis indicates the normalized token length (token count divided by the maximum allowed length), and the vertical axis represents the probability density. Blue curves correspond to our method (DRP), and orange curves denote the baseline. The reduction in long-tail completions and high-token outliers indicates that DRP mitigates verbose and degenerate reasoning failures, resulting in more robust and efficient inference.

Fine-grained supervised pruning improves reasoning accuracy more effectively than RL-based methods. ThinkPrune demonstrates strong performance on the 1.5B model, suggesting that pruning is effective in eliminating distracting or redundant reasoning steps, particularly for smaller-capacity models. Notably, it achieves solid gains on benchmarks like AIME24 and AMC. However, our DRP method yields higher accuracy improvements, likely due to its use of high-quality teacher-guided pruning. While ThinkPrune achieves slightly better compression on a few tasks—due to its explicit optimization for token length, DRP achieves notably higher accuracy across the board, while still maintaining competitive compression rates.

5.3 Token Usage Distribution Shift

Figure 3 shows the distribution of normalized token lengths before and after applying DRP across all benchmarks using R1-Distill-Qwen-7B. Detailed observations are discussed below:

Concise Reasoning Across Tasks. DRP effectively compresses the overall reasoning length across all benchmarks. The main density of the token distribution shifts leftward, especially on harder out-of-domain tasks like MATH500, AIME24, and AMC. This demonstrates that DRP encourages more concise reasoning behavior.

Eliminating Verbose and Degenerate Reasoning Failures. DRP significantly reduces long-tail completions that typically result from verbose or degenerate reasoning. On AMC, the baseline exhibits a bimodal distribution, where the secondary peak reflects unnecessarily lengthy yet still valid reasoning paths. More critically, we observe a clear drop in density near the upper token limit—especially in AIME and AMC—indicating that DRP mitigates degenerate cases where the model previously generated excessively long or looping outputs due to failure to converge (Section 4.6). This results in improved *robustness*, defined here as the model's ability to terminate reasonably when it cannot reach a correct solution.

Further Compression of Already Efficient Reasoning Paths. On datasets such as GSM8K and MATH500, where baseline models already produce relatively short completions, DRP still yields measurable compression gains. This indicates that DRP not only removes verbosity but also optimizes reasoning even in high-performing regimes.

6 Ablation Studies

We conduct three ablation experiments to understand the contribution of each component in our DRP framework separately.

6.1 Skill-based Decomposition vs. Default Step Split

RQ1: Does skill-based decomposition improve downstream learning compared to default step segmentation?

To assess the impact of our skill-based decomposition on downstream performance, we compare to a baseline that applies a default step-wise split without leveraging any functional skill information.

Dataset	Method	Accuracy	Tokens
GSM8K	7B Base	91.7%	917
	Default	92.7%	350
	DRP	94.1%	328
OOD Tasks			
MATH500	7B Base	92.4%	2486
	Default	92.0%	1905
	DRP	93.0%	1781
AIME24	7B Base	15/30	8674
	Default	14/30	4678
	DRP	15/30	4966
AMC	7B Base	31/40	4845
	Default	31/40	4975
	DRP	33/40	3258

Table 2: Comparison of 7B base model, default step segmentation, and our DRP method across in-distribution and out-of-distribution math benchmarks.

Both variants are followed by teacher-guided revision and used to train models within the same DRP framework.

Comparison of Decomposition Prompts

Skill-Based Prompt: Segment the explanation into a sequence of clear, non-overlapping steps, where each step corresponds to exactly one atomic mathematical skill. These skills may include, for example, *addition*, *subtraction*, *applying a formula*, *interpreting a quantity*, *simplifying*, *checking a condition*, and so on.

Default Prompt: Segment the explanation into clear, non-overlapping steps.

As shown in Table 2, while default segmentation already provides some benefit under the self-revision setup, our skill-based segmentation consistently achieves stronger results across all benchmarks. DRP with skill-aware segmentation not only improves accuracy (e.g., +2 points on AMC) but also reduces token usage more effectively (e.g., 3258 vs. 4975 tokens on AMC).

This performance gap can be attributed in part to the difference in decomposition granularity: on the GSM8K training set, our method produces an average of 12.6 steps per example, compared to only 8.3 steps under the default strategy (see Appendix E for an illustrative example). The finer-grained, skill-aligned structure facilitates more precise pruning and better alignment with distinct math operations.

Crucially, this structural advantage also trans-

lates into improved generalization: we observe that the benefit of skill-based segmentation holds even on out-of-distribution reasoning tasks, further demonstrating its role in producing stable and transferable reasoning supervision.

6.2 Structured Pruning vs. Direct Distillation

RQ2: Does the performance gain come from shorter CoTs or the structured pruning process?

To answer this, we compare DRP with a direct distillation baseline, where the student model is trained on concise CoT generated directly by GPT-4o. These responses are notably short—averaging **186 tokens**—compared to DRP's CoT average of **330 tokens**. Table 4 presents the results.

We observe that:

- On the in-distribution dataset GSM8K, direct distillation reduces token usage by over 50% (425 vs. 917 tokens) with only a minor accuracy drop (90.7% vs. 91.7%), indicating its effectiveness in simpler tasks.
- On OOD tasks like MATH500, AIME24, and AMC, however, accuracy drops significantly despite only modest token savings (e.g., 88.6% vs. 93.0% on MATH500; 13/30 vs. 15/30 on AIME24), suggesting limited generalization.

These findings suggest that shorter CoT alone do not guarantee better generalization in reasoning tasks. While direct distillation from teacher LLMs provides compressed outputs, these often omit essential intermediate reasoning steps. In contrast, DRP explicitly removes redundant content while preserving a step-level, skill-aligned structure, allowing the model to internalize more transferable problem-solving patterns.

This highlights an important insight: Retaining structurally meaningful reasoning—even at the cost of moderate length—yields superior downstream performance compared to blindly imitating shorter outputs. A comparison between short-form and DRP-style trajectories is in Appendix F.

6.3 Influence of Teacher Model Choice

RQ3: How does the choice of teacher model influence pruning effectiveness?

To evaluate the sensitivity of our pruning framework to the choice of teacher model, we experiment with three different large language models (LLMs): GPT-4o, Gemini 2.0 Flash, and ChatGPT. As shown in Table 3, all teacher models lead to consistent improvements in both accuracy and token efficiency compared to the base model, confirming the robustness and generalizability of our approach.

Among them, GPT-4o yields the strongest performance across most benchmarks, particularly in token compression. However, the overall differences between teacher models remain modest, suggesting that while stronger teachers can produce slightly more optimal pruning decisions, our method remains effective even when relying on weaker or more accessible models.

This indicates that our framework does not overfit to a specific teacher's internal reasoning patterns, and can generalize pruning supervision across a variety of LLMs. Notably, we observe that variations across teachers are more pronounced in token usage than in accuracy—highlighting that our method's core benefit, skill-level reasoning distillation, remains stable regardless of teacher strength.

7 Conclusion

We propose skill-based Distilled Reasoning Pruning, a framework that leverages pruned reasoning traces to distill smaller reasoning models. It outperforms direct distillation by effectively bridging the learnability gap between student and teacher models. The success of our skill-based step decomposition underscores the importance of fine-grained, consistent step segmentation as a strong foundation for pruning.

8 Limitations

While our method demonstrates strong performance on existing small-scale reasoning models, it remains uncertain how well it generalizes to other architectures. Currently, there is a limited number of publicly available small-sized LRMs with strong reasoning capabilities, making broad validation challenging. In addition, the training paradigms for reasoning models are evolving rapidly, and it is unclear whether the overthinking and inefficiency issues we target will persist in future model generations. We view our work as a step toward addressing current bottlenecks, but acknowledge that its relevance may shift as the landscape of LLM training continues to change.

9 Ethics

Our study uses the OpenAI and Google Gemini API on experiments above, and at no point do we access, or attempt to access, the true training data

Method	GSM8K		MATH500		AIME24		AMC	
	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens
7B-base student model	91.7%	917	92.4%	2486	15/30	8674	31/40	4845
with GPT-4o teacher	94.1%	328	93.0%	1781	15/30	4966	33/40	3258
with Gemini 2.0 flash teacher	93.2%	419	91.8%	2245	14/30	4860	33/40	2835
with ChatGPT teacher	91.2%	386	90.2%	2122	14/30	5265	32/40	3101

Table 3: Impact of different teacher models on DRP performance. We compare GPT-4o, Gemini 2.0 Flash, and ChatGPT as pruning teachers, evaluating downstream accuracy and average token usage on four math benchmarks.

Dataset	Method	Accuracy	Tokens
GSM8K	7B Base	91.7%	917
	Distill	90.7%	425
	DRP	94.1%	328
OOD Tasks			
MATH500	7B Base	92.4%	2486
	Distill	88.6%	2152
	DRP	93.0%	1781
AIME24	7B Base	15/30	8674
	Distill	13/30	6417
	DRP	15/30	4966
AMC	7B Base	31/40	4845
	Distill	28/40	4279
	DRP	33/40	3258

Table 4: Comparison between 7B base model, direct distillation from GPT-4o, and our DRP method across both in-distribution and OOD math benchmarks.

behind these models, or any underlying components of the systems. **Risks** The several datasets in our experiment are sourced from publicly available sources. However, we cannot guarantee that they are devoid of socially harmful or toxic language. We use ChatGPT [4] to correct grammatical errors in this paper.

References

AI-MO Team. 2024a. AIMO Validation Set - AIME Subset. https://huggingface.co/datasets/AI-MO/aimo-validation-aime.

AI-MO Team. 2024b. AIMO Validation Set - AMC Subset. https://huggingface.co/datasets/AI-MO/aimo-validation-amc.

Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2025. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567*.

Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. 2024. Do not think that much for 2+ 3=? on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. 2025. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*.

Google DeepMind. 2025. Introducing gemini 2.0 flash. https://blog.google/technology/google-deepmind/gemini-model-updates-february-2025/.

Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. 2024. Efficiently serving llm reasoning programs with certaindex. *arXiv preprint arXiv:2412.20993*.

Yichao Fu, Junda Chen, Yonghao Zhuang, Zheyu Fu, Ion Stoica, and Hao Zhang. 2025. Reasoning without self-doubt: More efficient chain-of-thought through certainty probing. In *ICLR 2025 Workshop on Foundation Models in the Wild*.

Mingqi Gao, Jie Ruan, Renliang Sun, Xunjian Yin, Shiping Yang, and Xiaojun Wan. 2023. Human-like summarization evaluation with chatgpt. *arXiv preprint arXiv:2304.02554*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2024. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.

[4] https://chatgpt.com/

Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. 2025. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning. *arXiv preprint arXiv:2504.01296*.

Yuxuan Jiang and Francis Ferraro. 2024. Memorization over reasoning? exposing and mitigating verbatim memorization in large language models' character understanding evaluation. *arXiv preprint arXiv:2412.14368*.

Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, et al. 2024. From generation to judgment: Opportunities and challenges of llm-as-a-judge. *arXiv preprint arXiv:2411.16594*.

Dawei Li, Renliang Sun, Yue Huang, Ming Zhong, Bohan Jiang, Jiawei Han, Xiangliang Zhang, Wei Wang, and Huan Liu. 2025a. Preference leakage: A contamination problem in llm-as-a-judge. *arXiv preprint arXiv:2502.01534*.

Yuetai Li, Xiang Yue, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, Bhaskar Ramasubramanian, and Radha Poovendran. 2025b. Small models struggle to learn from strong reasoners. *arXiv preprint arXiv:2502.12143*.

Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, et al. 2025c. From system 1 to system 2: A survey of reasoning large language models. *arXiv preprint arXiv:2502.17419*.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. 2025. Cot-valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2502.09601*.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*.

Tergel Munkhbat, Namgyu Ho, Seo Hyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. 2025. Self-training elicits concise reasoning in large language models. *arXiv preprint arXiv:2502.20122*.

OpenAI. 2022. Chatgpt: Openai's conversational ai. https://chat.openai.com/.

OpenAI. 2024a. Gpt-4o: Openai's multimodal model. https://openai.com/index/hello-gpt-4o/.

OpenAI. 2024b. https://openai.com/index/learning-to-reason-with-llms/.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, et al. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*.

Yongqi Tong, Dawei Li, Sizhe Wang, Yujia Wang, Fei Teng, and Jingbo Shang. 2024. Can llms learn from previous mistakes? investigating llms' errors to boost for reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3065–3080.

Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. 2025. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*.

Fengli Xu, Qianyue Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, et al. 2025a. Towards large reasoning models: A survey of reinforced reasoning with large language models. *arXiv preprint arXiv:2501.09686*.

Jingxian Xu, Mengyu Zhou, Weichang Liu, Hanbing Liu, Shi Han, and Dongmei Zhang. 2025b. Twt: Thinking without tokens by habitual reasoning distillation with multi-teachers' guidance. *arXiv preprint arXiv:2503.24198*.

Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, and Radha Poovendran. 2024. Stronger models are not stronger teachers for instruction tuning. *arXiv preprint arXiv:2411.07133*.

Yiyao Yu, Yuxiang Zhang, Dongdong Zhang, Xiao Liang, Hengyuan Zhang, Xingxing Zhang, Ziyi Yang, Mahmoud Khademi, Hany Awadalla, Junjie Wang, et al. 2025. Chain-of-reasoning: Towards unified mathematical reasoning in large language models via a multi-paradigm perspective. *arXiv preprint arXiv:2501.11110*.

Zhiyuan Zeng, Qinyuan Cheng, Zhangyue Yin, Yunhua Zhou, and Xipeng Qiu. 2025. Revisiting the test-time scaling of o1-like models: Do they truly possess test-time scaling capabilities? *arXiv preprint arXiv:2502.12215*.

Xunyu Zhu, Jian Li, Can Ma, and Weiping Wang. 2024. Improving mathematical reasoning capabilities of small language models via feedback-driven distillation. *arXiv preprint arXiv:2411.14698*.

A Outliers and Cutting Off

We can observe from Figure 4 that most answers are captured by 11K tokens, though some require up to 38K, revealing a long-tail pattern.

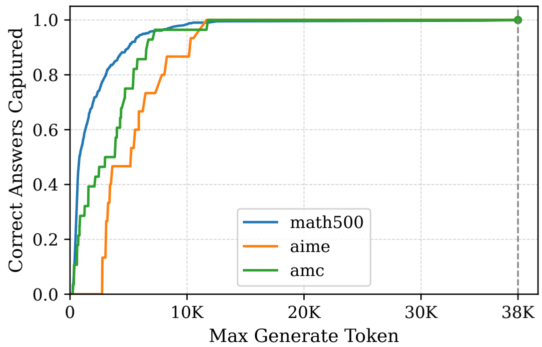

Figure 4: The x-axis denotes the model's maximum generation length, and the y-axis shows the proportion of correct answers recovered within that budget, with the R1-Distill-Qwen-7B.

As shown in Table 5, applying the 12k token cutoff has a significant effect, particularly on harder benchmarks like AIME24 and AMC. On these tasks, the model occasionally fails to generate valid answers and enters degenerate loops, repeatedly producing the same text until reaching the maximum token limit. As a result, the average token usage can increase by up to 5×, which does not reflect the true distribution of reasoning length and severely skews efficiency comparisons.

B Fine-Tuning Details

We fine-tune the DeepSeek-R1-Distill-Qwen models (7B and 1.5B) using the LLAMA-FACTORY framework with LoRA adaptation. The training data includes 8,000 samples drawn from GSM8K and PRM12K. We set a cutoff length of 4096 tokens for both input and output sequences.

The models are fine-tuned for 3 epochs using the following configuration:

- **Cutoff length:** 4096
- **Max samples:** 8000
- **Batch size:** 2 (with gradient accumulation of 4)
- **Learning rate:** 3e-5 with cosine schedule
- **Precision:** bf16
- **Validation split:** 5% of training data
- **Evaluation strategy:** every 300 steps

Training is performed using two A100 80GB GPU. All experiments use `overwrite_cache=true` and 8 parallel preprocessing workers. The resulting models are directly used for downstream evaluation without additional tuning.

C Prompt Templates

The full prompt for skill-based step segmentation:

Skill-Based Step Segmentation Prompt (Full Version)

You are given a complete reasoning path for a math problem. Your task is to segment it into a sequence of clear, non-overlapping steps, where each step corresponds to exactly one atomic mathematical skill. These skills may include, for example, addition, subtraction, applying a formula, interpreting a quantity, simplifying, checking a condition, and so on.
Use the following format for each step:
```
Step n: {{original text segment}}
Skill: {{name of the skill used}}
```
Only segment and label the steps — do not solve or modify the original content in any way.

The full prompt for self-revision:

Method	GSM8K		MATH500		AIME24		AMC	
	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens	Accuracy	Tokens
Baseline (no cutoff)	91.7%	917	92.5%	5435	15/30	54339	32/40	18736
Baseline (12k cutoff)	91.7%	917	92.4%	2486	15/30	8674	31/40	4845

Table 5: Impact of applying a 12k token cutoff on measured token usage across benchmarks. Accuracy remains largely unaffected, while the average token count drops significantly—particularly on harder tasks like AIME and AMC. This demonstrates the necessity of outlier mitigation for fair efficiency comparison.

Self-Revision Prompt (Full Version)

You are an expert in mathematical reasoning compression.
Given a list of reasoning steps labeled with their respective skills, your task is to evaluate and revise each step according to one of the following four actions:
*1. **KEEP**: The step is necessary and already concise. Keep it unchanged.*
*2. **DELETE**: The step is unnecessary and should be removed entirely.*
*3. **SINGLE-STEP COMPRESS**: The step is necessary but verbose; rewrite it in a more concise way.*
*4. **MULTI-STEP COMPRESS**: The step can be merged with neighboring steps; write a combined, cleaner version.*
If the final step clarifies the final answer (e.g., "The answer is..."), retain it.
After completing the revision of each step, synthesize the revised steps into a coherent explanation. Ensure the output is fluent, logically sound, and matches the speaker's tone and style.

D Skill Base Decomposition Evaluation

To assess the quality of our skill-based step decomposition, we conduct a pairwise comparison experiment using **Gemini 2.0 Flash** as the judge (Gao et al., 2023; Li et al., 2024, 2025a). For each reasoning trace T, we generate two segmentations: (1) our proposed *skill-based decomposition*, and (2) a baseline *sentence-based split* (obtained by naïvely splitting at punctuation or sentence boundaries).

We prompt Gemini to select the version with more **semantically coherent**, **structurally stable**, and **granularity-appropriate** steps, using the following instruction:

> Given two versions of step-by-step reasoning derived from the same original trace, please choose the version that shows better semantic coherence, structural consistency, and appropriate granularity. Respond with 'A', 'B', or 'Same'.

Among 50 randomly sampled traces from GSM8K, Gemini selected our skill-based decomposition in **33** cases, while preferring the baseline in only **17** cases. This result suggests that our segmentation method provides more interpretable and structurally aligned reasoning steps, which are beneficial for downstream pruning and distillation.

E Skill-based Decomposition vs. Default

In this section, we show a non-cherry picking example of how our skill based decomposition different from the default one. As shown in Table 6, our method has a more fine-grained step decomposition effect which leads to a more detailed pruning effect.

F Long CoT vs. Short CoT

In this section, we show a non-cherry picking example of how our skill based decomposition different from the default one. As shown in Table 7, our DRP-style CoT are more informative. That might because of LLM's unfair ability on different aspects (Jiang and Ferraro, 2024).

Form	Decomposition Output
Problem	Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?
Original Long-form CoT	First, I need to determine Joy's reading rate. She reads 8 pages in 20 minutes, so her reading rate is 8 pages per 20 minutes. Next, I'll calculate how many pages she reads per minute by dividing 8 pages by 20 minutes, which gives 0.4 pages per minute. Then, to find out how many minutes it takes her to read 120 pages, I'll divide 120 pages by her reading rate of 0.4 pages per minute, resulting in 300 minutes. Finally, I'll convert 300 minutes into hours by dividing by 60, which equals 5 hours.
DRP Decomposition (Skill-Based)	**Step 1:** Determine Joy's reading rate. She reads 8 pages in 20 minutes, so her reading rate is 8 pages per 20 minutes. *Skill: Interpreting a quantity* **Step 2:** Calculate how many pages she reads per minute by dividing 8 pages by 20 minutes. *Skill: Division* **Step 3:** The result of the division is 0.4 pages per minute. *Skill: Simplifying* **Step 4:** To find out how many minutes it takes her to read 120 pages, divide 120 pages by 0.4 pages per minute. *Skill: Division* **Step 5:** The result of the division is 300 minutes. *Skill: Simplifying* **Step 6:** Convert 300 minutes into hours by dividing by 60. *Skill: Division* **Step 7:** The result of the division is 5 hours. *Skill: Simplifying*
DRP Pruning and Final Result	1. **KEEP:** The step is necessary and already concise. 2. **SINGLE-STEP COMPRESS:** Joy reads 8 pages in 20 minutes, so she reads 0.4 pages per minute ($8 \div 20$). 3. **DELETE:** This step is now included in the previous step. 4. **SINGLE-STEP COMPRESS:** To find the time for 120 pages, divide 120 by 0.4 pages per minute. 5. **DELETE:** This step is now included in the previous step. 6. **SINGLE-STEP COMPRESS:** Convert 300 minutes to hours by dividing by 60. 7. **DELETE:** This step is now included in the previous step. **Final Answer:** Okay, let's calculate the time it takes Joy to read 120 pages. Joy reads 8 pages in 20 minutes, which means she reads 0.4 pages per minute ($8 \div 20$). To find out how many minutes it takes her to read 120 pages, divide 120 by 0.4, resulting in 300 minutes. Convert 300 minutes into hours by dividing by 60, which gives us 5 hours. The answer is 5 hours.
Default Decomposition	**Step 1:** First, I need to determine Joy's reading rate. She reads 8 pages in 20 minutes, so her reading rate is 8 pages per 20 minutes. **Step 2:** Next, I'll calculate how many pages she reads per minute by dividing 8 pages by 20 minutes, which gives 0.4 pages per minute. **Step 3:** Then, to find out how many minutes it takes her to read 120 pages, I'll divide 120 pages by her reading rate of 0.4 pages per minute, resulting in 300 minutes. **Step 4:** Finally, I'll convert 300 minutes into hours by dividing by 60, which equals 5 hours.
Default Pruning and Final Result	**Step 1:** SINGLE-STEP COMPRESS Revised: Joy's reading rate is 8 pages per 20 minutes. **Step 2:** SINGLE-STEP COMPRESS Revised: Dividing 8 by 20, Joy reads 0.4 pages per minute. **Step 3:** SINGLE-STEP COMPRESS Revised: Dividing 120 by 0.4, it takes her 300 minutes to read 120 pages. **Step 4:** SINGLE-STEP COMPRESS Revised: Converting 300 minutes to hours: $300 \div 60 = 5$ hours. **Final Answer:** Joy's reading rate is 8 pages per 20 minutes. Dividing 8 by 20, Joy reads 0.4 pages per minute. Dividing 120 by 0.4, it takes her 300 minutes to read 120 pages. Converting 300 minutes to hours: $300 \div 60 = 5$ hours. **The answer is 5.**

Table 6: A comparison between DRP's skill-based decomposition and default segmentation with compression, for the same math problem.

Form	Reasoning Trajectory
Problem: Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?	
Original Long-form CoT	First, I need to determine Joy's reading rate. She reads 8 pages in 20 minutes, so her reading rate is 8 pages per 20 minutes. Next, I'll calculate how many pages she reads per minute by dividing 8 pages by 20 minutes, which gives 0.4 pages per minute. Then, to find out how many minutes it takes her to read 120 pages, I'll divide 120 pages by her reading rate of 0.4 pages per minute, resulting in 300 minutes. Finally, I'll convert 300 minutes into hours by dividing by 60, which equals 5 hours.
DRP-style (Ours)	Okay, let's calculate the time it takes Joy to read 120 pages. Joy reads 8 pages in 20 minutes, which means she reads 0.4 pages per minute ($8 \div 20$). To find out how many minutes it takes her to read 120 pages, divide 120 by 0.4, resulting in 300 minutes. Convert 300 minutes into hours by dividing by 60, which gives us 5 hours. The answer is 5 hours.
Short-form CoT	In one hour, there are 3 sets of 20 minutes. So, Joy can read $8 \times 3 = 24$ pages in an hour. It will take her $120 \div 24 = 5$ hours to read 120 pages.

Table 7: A comparison between original long-form, DRP-style, and short-form reasoning traces for the same math problem.